

Embedding Spaces and Learned Representations

Mathematics of Deep Learning — Chapter 8

Yin Yang

Foundation Books Project

Roadmap

Embedding Matrices and Lookup

Geometry: Similarity and Distance

Pooling Word Vectors to Review Vectors

Classification on Top of Embeddings

How Losses Shape Geometry

Pretrained Embeddings: Freeze, Finetune, Anchor

Evaluating and Debugging Embeddings

Shared Multimodal Embedding Spaces

Summary

Embedding Matrices and Lookup

From Discrete Objects to Vectors

Embeddings turn discrete objects — words, users, products, images — into vectors. Their value comes from *geometry*, not from the coordinate list alone.

- Running example: small movie-review sentiment task.
- Tokenized review $x = (x_1, \dots, x_T)$, $x_t \in \mathcal{V}$.
- Each token has an integer ID $\iota(x_t) \in \{1, \dots, m\}$.
- The ID is an *address*, not a measurement.

Workflow: review $\rightarrow$ IDs $\rightarrow$ word vectors $\rightarrow$ review vector $\rightarrow$ score.

Useful embeddings put *excellent* near *great* and far from *terrible* — if the task rewards it.

One-Hot Geometry Encodes Only Identity

For one-hot vectors $e_i, e_j \in \mathbb{R}^m$,

$$e_i^\top e_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j, \end{cases} \quad \|e_i - e_j\|_2^2 = \begin{cases} 0, & i = j, \\ 2, & i \neq j. \end{cases}$$

Every pair of distinct tokens is equally far apart.

- Vocabulary of size 50,000 $\Rightarrow$ 50,000-dim sparse vector.
- No graded notion of semantic closeness.
- Token IDs avoid false numerical order but lose structure.

One-hot says “same or different” — nothing more.

Embedding Matrix and Lookup, Step by Step

token
excellent

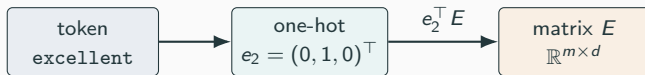
(1) The token is a discrete object.

Embedding Matrix and Lookup, Step by Step



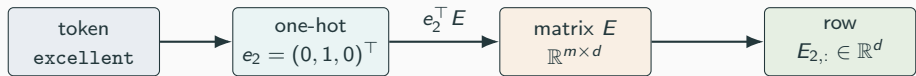
- (1) The token is a discrete object.
- (2) Its ID names a position; one-hot is the abstract address.

Embedding Matrix and Lookup, Step by Step



- (1) The token is a discrete object.
- (2) Its ID names a position; one-hot is the abstract address.
- (3) Lookup is matrix multiplication: $e_i^\top E = E_{i,:}$.

Embedding Matrix and Lookup, Step by Step



- (1) The token is a discrete object.
- (2) Its ID names a position; one-hot is the abstract address.
- (3) Lookup is matrix multiplication: $e_i^\top E = E_{i,:}$.
- (4) Libraries implement this as row selection (faster).

Lookup is one-hot matrix multiplication, executed as row indexing.

Shapes and the Parameter Ledger

Object	Shape	Role
Embedding matrix E	$m \times d$	vocab rows
Token IDs (batch)	$B \times T$	integer addresses
Embedded batch	$B \times T \times d$	after lookup
Pooled review vector r	$B \times d$	one per example
Classifier weights w	d	one head

Example ledger: $m = 10,000$, $d = 50 \Rightarrow 500,000$ trainable scalars in E alone. Batch of $B = 32$, $T = 80$ tokens gives an embedded tensor of shape $32 \times 80 \times 50$.

Sequence length T , batch B , vocab m , dim d play four distinct roles — many bugs are shape confusions.

Lookup Gradients Are Sparse

Let $S \subseteq \{1, \dots, m\}$ be the token IDs in a minibatch. For any $j \notin S$:

$$\frac{\partial L}{\partial E_{j,:}} = 0.$$

Only the rows actually used by the batch get a direct gradient.

- Embedding lookup is exploited by libraries: sparse updates.
- Tokens unseen in a batch do not move (from data loss alone).
- Optimizer state and regularization may still touch them.

A minibatch trains the rows it touches — nothing else receives a direct lookup gradient.

Geometry: Similarity and Distance

Three Comparison Rules

For $u, v \in \mathbb{R}^d$:

$$u^\top v = \sum_j u_j v_j, \quad \cos(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}, \quad d_2(u, v) = \|u - v\|_2.$$

- **Dot product:** grows with direction *and* length.
- **Cosine:** length-invariant, compares direction only.
- **Euclidean distance:** compares absolute position.

An embedding space is coordinates *plus* a comparison rule — changing the rule can change which neighbors are closest.

Unit-Normalization Aligns the Rules

If $\|u\|_2 = \|v\|_2 = 1$:

$$\cos(u, v) = u^\top v, \quad \|u - v\|_2^2 = 2 - 2u^\top v.$$

Among unit-norm candidates:

- maximizing cosine = maximizing dot product,
- = minimizing Euclidean distance,
- all give the *same* nearest-neighbor ranking.

Direction example: $u = (1, 2)$, $v = (2, 4)$, $w = (2, 0)$. Then $\cos(u, v) = 1$ while $\cos(u, w) < 1$.
Cosine separates direction from length.

Normalize first if you want the comparison rules to agree.

Johnson–Lindenstrauss: Distances Compress

For any $0 < \epsilon < 1$ and any n points, there exists a map into $\mathbb{R}^k$ with

$$k = O\left(\frac{\log n}{\epsilon^2}\right)$$

such that every pairwise distance is preserved within $(1 \pm \epsilon)$.

- Finite-cloud guarantee — not about future inputs.
- Distances in a finite set can live in much fewer dimensions.
- Justifies low-dim embeddings for retrieval, in geometry only.

The guarantee is about preserved *distances*, not preserved *meaning*.

Pooling Word Vectors to Review Vectors

Pooling: From Token Lists to One Vector

Lookup produces a list $z_1, \dots, z_T \in \mathbb{R}^d$. Pooling gives one fixed-size vector $r \in \mathbb{R}^d$.

$$r_{\text{sum}} = \sum_{t=1}^T z_t, \quad r_{\text{mean}} = \frac{1}{T} \sum_{t=1}^T z_t, \quad r_{\text{weighted}} = \sum_{t=1}^T \alpha_t z_t.$$

Two-word example: $z_1 = (1, 1)$, $z_2 = (0, 2)$. Mean pool: $r = (0.5, 1.5)$.

- Mean pooling: simple, stable, hand-computable.
- Strong text-classification baseline (fastText).
- Cost: discards information not in the average.

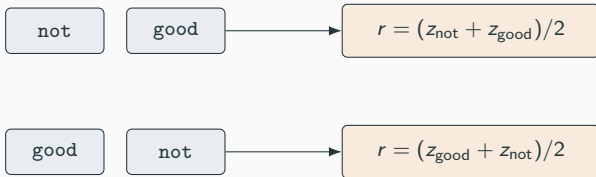
Pooling buys a fixed-size vector by giving up some sequence structure.

Permutation Invariance: A Built-In Limitation



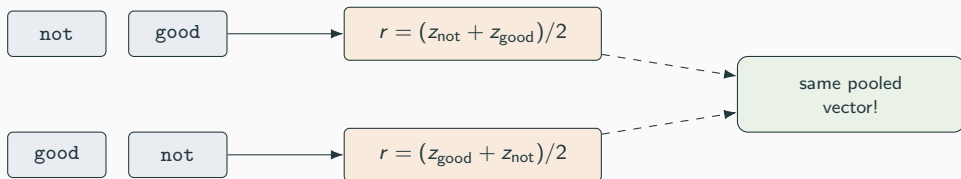
(1) Token order "not good".

Permutation Invariance: A Built-In Limitation



- (1) Token order "not good".
- (2) Token order "good not".

Permutation Invariance: A Built-In Limitation



- (1) Token order "not good".
- (2) Token order "good not".
- (3) Identical pooled r : bag-of-embeddings cannot see order.

Sequence models and attention exist to recover what pooling discards.

Classification on Top of Embeddings

Logistic Sentiment Model

After pooling, $r \in \mathbb{R}^d$. Score, probability, loss:

$$s = w^\top r + b, \quad \hat{p} = \sigma(s) = \frac{1}{1 + \exp(-s)},$$

$$\ell(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

Hand example: $r = (2, -1)$, $w = (1, 3)$, $b = -1 \Rightarrow s = -2$. With $y = 1$: $\hat{p} \approx 0.119$, $\ell \approx 2.13$.

- Score derivative: $\partial \ell / \partial s = \hat{p} - y$.
- Review-vector gradient: $\nabla_r \ell = (\hat{p} - y) w$.

The classifier weight w becomes the direction in which the loss pushes the review vector.

Gradient Flows into the Embedding Rows

With mean pooling and token j appearing n_j times in a review:

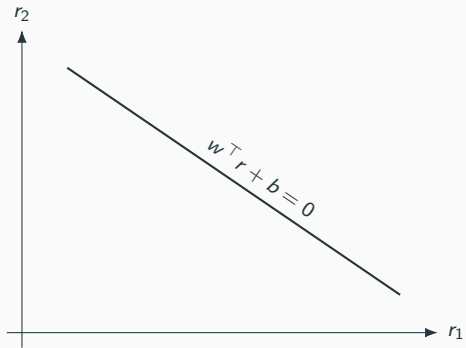
$$\frac{\partial \ell}{\partial E_{j,:}} = \frac{n_j}{T} (\hat{p} - y) w.$$

Tokens absent from the review: zero gradient.

- Only *observed* rows move per example.
- Repeated tokens amplify their update via n_j .
- Longer reviews dilute each occurrence by $1/T$.
- Direction along w depends on the sign of $\hat{p} - y$.

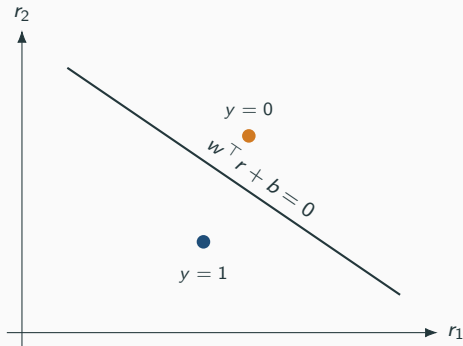
Supervised training reshapes the embedding rows the review actually used — local in the table.

Decision Boundary in Review-Vector Space



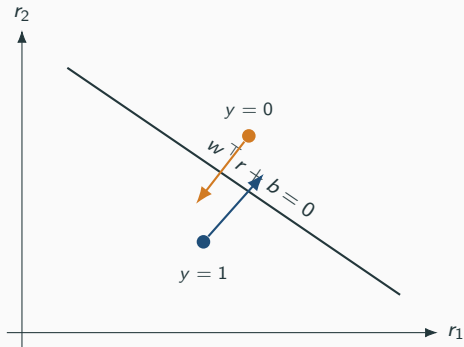
(1) Linear head defines a boundary.

Decision Boundary in Review-Vector Space



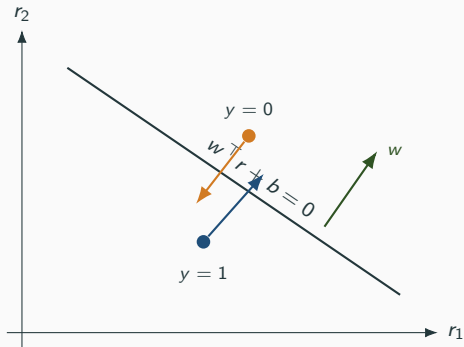
- (1) Linear head defines a boundary.
- (2) Positive and negative examples sit somewhere.

Decision Boundary in Review-Vector Space



- (1) Linear head defines a boundary.
- (2) Positive and negative examples sit somewhere.
- (3) Gradient updates push them to opposite sides.

Decision Boundary in Review-Vector Space



- (1) Linear head defines a boundary.
- (2) Positive and negative examples sit somewhere.
- (3) Gradient updates push them to opposite sides.
- (4) The direction is w (and $-w$ for negatives).

Supervised embeddings organize themselves around the labels the loss rewards.

How Losses Shape Geometry

Pair Labels Define “Similar”

Pair label $r(i, j) \in \{0, 1\}$: positive when $r(i, j) = 1$, negative otherwise. *The task picks the meaning.*

- Sentiment: reviews with same label are positive pairs.
- Image-caption: a true caption is the positive partner.
- Retrieval: query and a relevant document are a positive pair.

Same word, different geometries

excellent and terrible are both strong opinion words.

- Topic embedding: near each other (both “evaluation”).
- Sentiment embedding: far apart (opposite labels).

The loss tells the space what “close” is supposed to mean.

Triplet Margin Loss

Anchor a , positive p , negative n ; distance d ; margin $m > 0$:

$$\ell(a, p, n) = \max\{0, m + d(a, p) - d(a, n)\}.$$

Zero loss iff $d(a, n) \geq d(a, p) + m$. Example ($a = (0, 0)$, $p = (1, 0)$, $m = 0.5$):

- $n = (2, 0)$: $d(a, n) = 2$, $\ell = \max\{0, 0.5 + 1 - 2\} = 0$.
- $n = (1.2, 0)$: $\ell = \max\{0, 0.5 + 1 - 1.2\} = 0.3$.

Modern contrastive losses use many positives and negatives inside a softmax-like objective (InfoNCE).

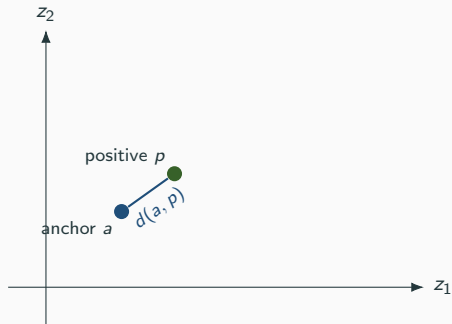
Pull matched objects together; push mismatched ones apart by at least a margin.

Triplet Geometry, Visualized



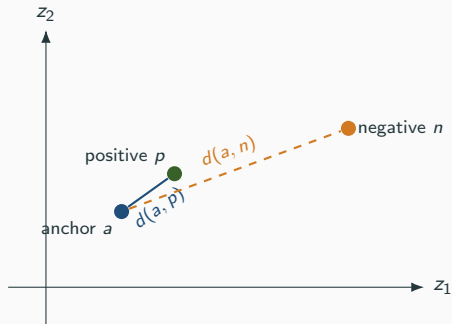
(1) Pick an anchor.

Triplet Geometry, Visualized



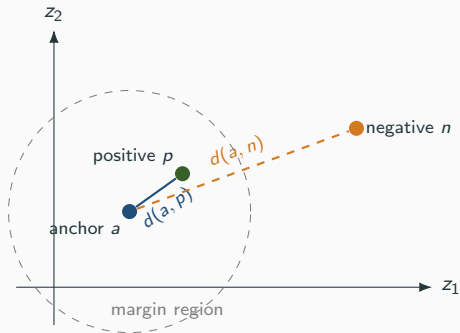
- (1) Pick an anchor.
- (2) Pull the positive close.

Triplet Geometry, Visualized



- (1) Pick an anchor.
- (2) Pull the positive close.
- (3) Push the negative away.

Triplet Geometry, Visualized



- (1) Pick an anchor.
- (2) Pull the positive close.
- (3) Push the negative away.
- (4) The margin m defines “how much farther”.

The objective *defines* the geometry — not the embedding dimension alone.

Pretrained Embeddings: Freeze, Finetune, Anchor

Three Ways to Reuse Pretrained E_0

Pretrained matrix $E_0 \in \mathbb{R}^{m \times d}$ carries source-task geometry. Three target-task optimization problems:

$$\text{frozen: } \min_{\theta} \hat{R}_T(E_0, \theta),$$

$$\text{fine-tuned: } \min_{E, \theta} \hat{R}_T(E, \theta), \quad E \leftarrow E_0,$$

$$\text{anchored: } \min_{E, \theta} \hat{R}_T(E, \theta) + \lambda \|E - E_0\|_F^2.$$

- Freezing is the constraint $E = E_0$.
- Fine-tuning removes that constraint.
- Anchoring puts a λ -priced pull back to E_0 .

Freeze, fine-tune, anchor are different optimization problems — not just implementation toggles.

Anchored Fine-Tuning and the Tradeoff

Embedding gradient of the anchored objective:

$$\nabla_E \hat{R}_T(E, \theta) + 2\lambda(E - E_0).$$

- First term: target task wants to move the rows.
- Second term: pretrained anchor pulls them back.
- $\lambda \rightarrow \infty$: behaves like freezing.
- $\lambda = 0$: ordinary fine-tuning.

Reading the three validation numbers:

$$\hat{R}_{\text{val}}(E \text{ random}), \hat{R}_{\text{val}}(E_0 \text{ frozen}), \hat{R}_{\text{val}}(E \text{ init from } E_0).$$

λ is the price the target task pays to deviate from pretrained geometry.

Evaluating and Debugging Embeddings

Precision at k : Ranking-Based Evaluation

Queries Q , candidates C , relevance sets $R(q) \subseteq C$. Ranking $\pi_{q,f,s}$ orders C by similarity. Then

$$\text{P@ } k(q; f, s) = \frac{1}{k} \sum_{j=1}^k \mathbf{1}\{\pi_{q,f,s}(j) \in R(q)\},$$

$$\text{meanP@ } k(f, s) = \frac{1}{|Q|} \sum_{q \in Q} \text{P@ } k(q; f, s).$$

Example: top-3 retrieved = (c_7, c_2, c_9) , $R(q) = \{c_2, c_5, c_7\} \Rightarrow \text{P@ } 3 = 2/3$.

The pair (f, s) — embedding plus rule — determines the ranking and therefore the score.

Diagnostics: Anisotropy and Hubness

Length bias of the dot product:

$$u^\top v = \|u\| \|v\| \cos(u, v).$$

Large norms can dominate rankings. Anisotropy proxy (average off-diagonal cosine):

$$\frac{1}{N(N-1)} \sum_{i \neq j} \cos(z_i, z_j).$$

Large positive values warn: many vectors point in nearly one direction. Hubness count:

$$h_k(c) = \sum_{q \in Q} \mathbf{1}\{c \in \{\pi_q(1), \dots, \pi_q(k)\}\}.$$

A few hubs appearing in everyone's top- k is a red flag.

Embedding bugs hide in geometry — inspect norms, angles, and neighbor frequencies.

Intrinsic vs. Extrinsic Evaluation

Intrinsic

Evaluates the geometry directly: nearest-neighbor precision, correlation with word-similarity benchmarks, retrieval rankings.

Extrinsic

Evaluates the downstream task: validation loss $\hat{R}_{\text{val}}(f, \theta) = \frac{1}{n} \sum_i \ell(h_\theta(f(x_i)), y_i)$.

- Plausible neighbors $\neq$ good task performance.
- Validation data must *not* choose the representation.
- t-SNE / UMAP suggest hypotheses — they are not the metric.

An embedding is trusted through the decisions its geometry induces, not through a 2D plot.

Shared Multimodal Embedding Spaces

Shared Embedding Space: Two Encoders, One Geometry

Two encoders, one target space:

$$f_{\text{image}} : \mathcal{X}_{\text{image}} \rightarrow \mathbb{R}^d, \quad f_{\text{text}} : \mathcal{X}_{\text{text}} \rightarrow \mathbb{R}^d.$$

Both are compared by the *same* similarity rule s .

- Matched image-caption pairs should land close.
- Mismatched pairs should land far.
- Cross-modal comparison becomes a vector operation.
- Scaled-up example: CLIP.

Multimodal alignment = train two encoders so their outputs live in one comparable space.

Batch Softmax (InfoNCE) Contrastive Loss

Normalized matched embeddings $(u_i, v_i)_{i=1}^B$, $s(u, v) = u^\top v$, temperature $\tau > 0$:

$$\ell_i = -\log \frac{\exp(s(u_i, v_i)/\tau)}{\sum_{j=1}^B \exp(s(u_i, v_j)/\tau)}.$$

- Numerator rewards matched similarity.
- Denominator pushes against in-batch mismatches.
- Other captions in the batch act as negatives.

Informal mutual information lower bound:

$$I(X; Y) \geq \log B - \mathcal{L}_{\text{NCE}}.$$

Larger (well-chosen) batches make the comparison more informative.

Negatives are not noise — they are the comparison problem.

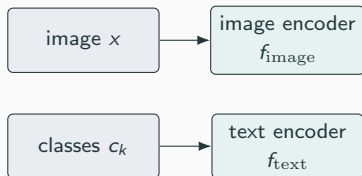
Zero-Shot Classification as Vector Comparison



(1) Encode the image once into the shared space.

Numerical check: $\cos(u, v_1) = 0.63$, $\cos(u, v_2) = 0.48 \Rightarrow$ predict class 1.

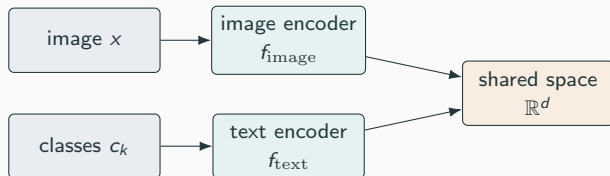
Zero-Shot Classification as Vector Comparison



- (1) Encode the image once into the shared space.
- (2) Encode each class description once into the same space.

Numerical check: $\cos(u, v_1) = 0.63$, $\cos(u, v_2) = 0.48 \Rightarrow$ predict class 1.

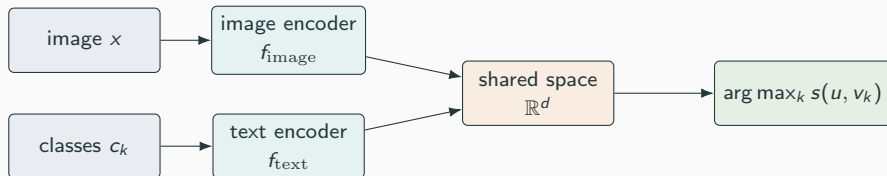
Zero-Shot Classification as Vector Comparison



- (1) Encode the image once into the shared space.
- (2) Encode each class description once into the same space.
- (3) Compare $u = f_{\text{image}}(x)$ with each $v_k = f_{\text{text}}(c_k)$.

Numerical check: $\cos(u, v_1) = 0.63$, $\cos(u, v_2) = 0.48 \Rightarrow$ predict class 1.

Zero-Shot Classification as Vector Comparison



- (1) Encode the image once into the shared space.
- (2) Encode each class description once into the same space.
- (3) Compare $u = f_{\text{image}}(x)$ with each $v_k = f_{\text{text}}(c_k)$.
- (4) Predict by argmax similarity — no new training needed.

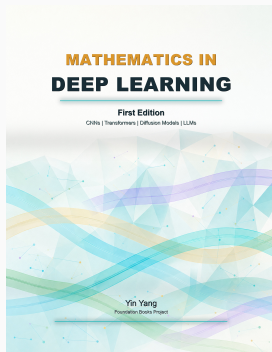
Numerical check: $\cos(u, v_1) = 0.63$, $\cos(u, v_2) = 0.48 \Rightarrow$ predict class 1.

Classification reduces to “which text vector is closest to the image vector”.

Summary

- **Lookup** is matrix multiplication on a one-hot: $e_i^\top E = E_{i,:}$. Token IDs are addresses.
- **Geometry** is coordinates *plus* a comparison rule (dot, cosine, distance).
- **Pooling** yields one fixed-size vector but loses sequence structure (mean pooling is permutation-invariant).
- **Classification** on r gives gradient $\nabla_r \ell = (\hat{p} - y)w$; mean pooling distributes it as $(n_j/T)(\hat{p} - y)w$ per used row.
- **Losses shape geometry**: triplet, contrastive objectives define what “close” means.
- **Pretrained** E_0 : freeze, fine-tune, or anchor with $\lambda \|E - E_0\|_F^2$.
- **Evaluation**: P@k, anisotropy, hubness; extrinsic validation loss is the bottom line.
- **Shared spaces**: two encoders, one geometry; zero-shot classification is argmax similarity.

**Next: sequence models, attention, and the architectures
that recover what pooling discards.**



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/mathematics-in-deep-learning
Code	github.com/foundation-books/mathematics-in-deep-learning-code
Paperback	amazon.com/dp/BOH1ZZHDT2
Hardcover	amazon.com/dp/BOH1ZL11MJ
Kindle	amazon.com/dp/B0GX32M8SP
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.